Projet INPF12 2024–2025

Éditeur de texte en OCaml

Mouhamed CISSE

03 mai 2025

1 Introduction

Ce projet consiste à concevoir un éditeur de texte simple en OCaml, basé sur la structure de données *zipper*. L'objectif est de manipuler un tampon (buffer) de lignes, chaque ligne étant elle-même un *zipper* de caractères, avec un curseur en plus d'un curseur.

2 Choix de conception

2.1 Structure `zipper`

Nous avons utilisé la structure de type zipper proposée :
— `before` : les éléments avant le curseur (dans l'ordre inverse),
— `current` : l'élément courant pointé par le curseur,
— `after` : les éléments après le curseur,
— `pos` : la position actuelle du curseur (int).

Cette structure permet de réaliser facilement et efficacement des déplacements et des insertions locales.

2.2 Tampon de texte

Le buffer est un `zipper` de lignes, où chaque ligne est elle-même un `zipper` de caractères. Cette façon de faire facilite les déplacements et modifications ligne par ligne, caractère par caractère.

3 Fonctionnalités implémentées

— Déplacements gauche/droite/haut/bas du curseur ;
— Insertion d'un caractère à la position du curseur ;
— Suppression (Delete) et retour arrière (Backspace) ;
— Insertion d'un saut de ligne.

Toutes ces actions sont réalisées grâce à des raccourcis clavier via l'interface graphique OCaml Graphics.

4 Implémentation des fonctions

4.1 Fonctions sur les zipper

`get_current : ('a, 'b) zipper -> 'b` Retourne l'élément courant dans un zipper.

`get_pos : buffer -> int` Retourne la position actuelle du curseur dans la ligne courante.

`fold : ('a -> 'c -> 'c) -> ('b -> 'c -> 'c) -> 'c -> ('a, 'b) zipper -> 'c` Fonction de fold personnalisée sur un zipper.

`update_with : (line -> line) -> buffer -> buffer` Applique une transformation à la ligne courante du buffer.

4.2 Fonctions de déplacement

`move_left : buffer -> buffer` Déplace le curseur d'un caractère vers la gauche.

`move_right : buffer -> buffer` Déplace le curseur d'un caractère vers la droite.

`move_up : buffer -> buffer` Monte le curseur à la ligne précédente, en conservant la colonne.

`move_down : buffer -> buffer` Descend le curseur à la ligne suivante, en conservant la colonne.

4.3 Fonctions de modification de texte

`insert_char : char -> buffer -> buffer` Insère un caractère à la position du curseur.

`do_suppr : buffer -> buffer` Supprime le caractère situé après le curseur.

`do_backspace : buffer -> buffer` Supprime le caractère juste avant le curseur.

`create_newline : buffer -> buffer` Insère une nouvelle ligne à la position courante.

5 Problèmes rencontrés et solutions

5.1 Gestion du zipper et du curseur

— **Problème :** Oubli que la liste `before` était inversée, entraînant des erreurs d'ordre des caractères. **Solution :** Porter une attention systématique à l'inversion dans tous les tests unitaires.

— **Problème :** Difficulté à savoir quelle liste (`before` ou `after`) modifier dans les fonctions de déplacement. **Solution :** Analyser précisément chaque cas et visualiser l'état du tampon après chaque opération.

5.2 Gestion du curseur en fin/début de ligne

— **Problème :** Position horizontale du curseur non conservée lors de déplacements verticaux. **Solution :** Ajout d'un ajustement de colonne après le swap de ligne.
— **Problème :** Lancement de l'interface graphique lors des tests unitaires. **Solution :** Commenter l'appel à `main` ou fermer l'interface (`Échap`) avant de lancer les tests.

6 Améliorations apportées

Pour les déplacements verticaux (`move_up` et `move_down`) :
— Calcul de la colonne cible à partir de la position initiale.
— Swap des listes `before`/`after` pour changer de ligne.
— Réajustement de la position horizontale en fonction de la longueur de la ligne cible.
Aucune amélioration spécifique n'a été apportée pour `do_suppr` et `do_backspace`.

7 Limites de l'implémentation

Notre éditeur de texte, bien que fonctionnel, présente encore certaines limitations :
— **Sélection et copier–coller** : non supportés.
— **Sauvegarde et chargement** : non supportés.
— **Interface graphique** : minimaliste, à moderniser.

8 Manuel d'utilisation

— `Ctrl-q / Ctrl-d / Ctrl-z / Ctrl-s` : déplacements du curseur.
— `Backspace` : supprime le caractère précédent.
— `Delete` : supprime le caractère courant.
— `Entrée` : insère un retour à la ligne.
— `Échap` : quitte l'éditeur.

9 Conclusion

Ce projet a permis de manipuler de manière approfondie les structures fonctionnelles telles que les zippers, et d'utiliser la bibliothèque graphique OCaml pour visualiser les opérations. Le code est facilement extensible pour ajouter de nouvelles fonctionnalités.

Fichiers fournis :
— `rapport.pdf` : ce rapport

— `projet.ml` : le code OCaml du projet
— `Makefile` : script de compilation
— `tests.ml` : tests unitaires